

Bases de Dados Relacionais e SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Panorama SQL e NoSQL

A Evolução das Bases de Dados

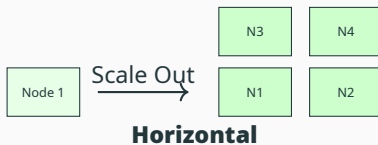
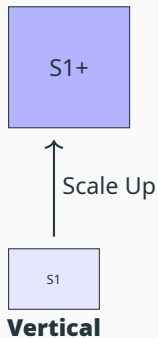
O mundo das bases de dados mudou de tabelas rígidas para um ecossistema diverso.

- **Anos 70-80:** RDBMS (Oracle, IBM) - Foco na eficiência de armazenamento.
- **Anos 90:** BDs Orientadas a Objetos - Breve tentativa de alinhar código e dados.
- **Anos 2000:** A Explosão do Big Data - Google/Amazon precisavam de escala massiva.
- **Anos 2010:** Revolução NoSQL - Foco em disponibilidade e esquemas flexíveis.
- **Presente:** Persistência Poliglota - Usar a ferramenta certa para a tarefa certa.

Relacional vs. Não-Relacional

- **Relacional (SQL):**
 - Esquema predeterminado (tabelas/linhas/colunas).
 - Consistência forte (ACID).
 - Ideal para dados estruturados e relações complexas.
- **Não-Relacional (NoSQL):**
 - Esquema dinâmico (documentos, pares, grafos).
 - Alta performance e escalonamento horizontal fácil.
 - Ideal para desenvolvimento rápido e conjuntos de dados massivos.

Escalonamento Vertical vs. Horizontal



- **ACID (Padrão SQL):**

- Atomicidade, Consistência, Isolamento, Durabilidade.
- Os dados estão sempre precisos e atualizados para todos os clientes.

- **BASE (Padrão NoSQL):**

- **B**asicamente **A**vailable (O sistema responde mesmo se nós falharem).
- **S**oft State (Os dados podem mudar sem input devido a regras de consistência).
- **E**ventual Consistency (Os dados serão consistentes... eventualmente).

Teorema CAP

Teorema CAP: Consistência

Consistência significa que todos os clientes veem os mesmos dados ao mesmo tempo, independentemente do nó a que se liguem.

- Para que isto aconteça, sempre que os dados são escritos num nó, devem ser replicados instantaneamente para todos os outros nós.
- Se ocorrer uma partição na rede, o sistema deve parar de aceitar escritas para garantir a consistência.

Teorema CAP: Disponibilidade

Disponibilidade significa que qualquer cliente que faça um pedido de dados recebe uma resposta, mesmo que um ou mais nós estejam em baixo.

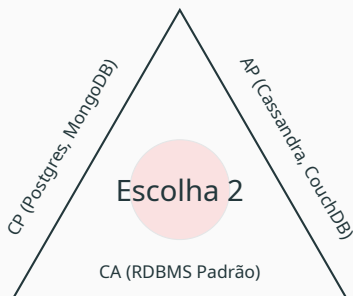
- Num sistema distribuído, isto é alcançado através de redundância.
- No entanto, se os nós não conseguirem comunicar (partição), podem devolver dados diferentes (antigos) para garantir que respondem sempre.

Teorema CAP: Tolerância à Partição

Tolerância à Partição significa que o sistema continua a operar apesar de um número arbitrário de mensagens serem perdidas (ou atrasadas) pela rede entre os nós.

- Nos sistemas distribuídos modernos, a **Tolerância à Partição não é negociável**. Falhas de rede acontecem.
- Portanto, os sistemas devem escolher entre **Consistência (CP)** ou **Disponibilidade (AP)** durante uma partição.

Tolerância Partição (P)



Consistência (C)

Disponibilidade (A)

Famílias NoSQL

As Quatro Famílias NoSQL

Para lidar com diferentes estruturas de dados e necessidades de escala, surgiram quatro famílias principais:

- **Key-Value Stores:** Simples, rápidas, dados opacos.
- **Document Stores:** Flexíveis, hierárquicas, dados semi-estruturados.
- **Wide-Column Stores:** Otimizadas para dados esparsos e de alto volume.
- **Graph Databases:** Otimizadas para relações complexas e travessias.

O modelo NoSQL mais simples. Os dados são armazenados como um valor opaco indexado por uma chave única.

- **Analogia:** Um Dicionário Python ou um HashMap Java.
- **Pontos Fortes:** Extremamente rápido, ótimo para dados simples.
- **Uso:** Gestão de sessões, preferências de utilizador, tabelas de classificação em tempo real.
- **Exemplo: Redis**, Amazon DynamoDB.

Exemplo de Uso Redis

Operações básicas

SET user:101 "Alice"

GET user:101 # Output: "Alice"

Incrementos atômicos

SET counter 10

INCR counter # Output: 11

Listas

LPUSH tasks "Email"

LPUSH tasks "Code"

RPOP tasks # Output: "Email"

Armazena dados em formato JSON, BSON ou XML. Os documentos são auto-descritivos.

- **Analogia:** Uma pasta de ficheiros JSON.
- **Pontos Fortes:** Esquema flexível (campos diferentes por documento), intuitivo para programadores.
- **Uso:** Gestão de conteúdos, catálogos de e-commerce, perfis de utilizador.
- **Exemplo: MongoDB, CouchDB.**

Exemplo de Uso MongoDB

```
// Inserir um documento
db.users.insertOne({
  name: "Alice",
  age: 25,
  skills: ["SQL", "Python"]
});

// Consultar com filtros flexíveis
db.users.find({ age: { $gt: 20 } });

// Atualizar campos específicos
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
);
```

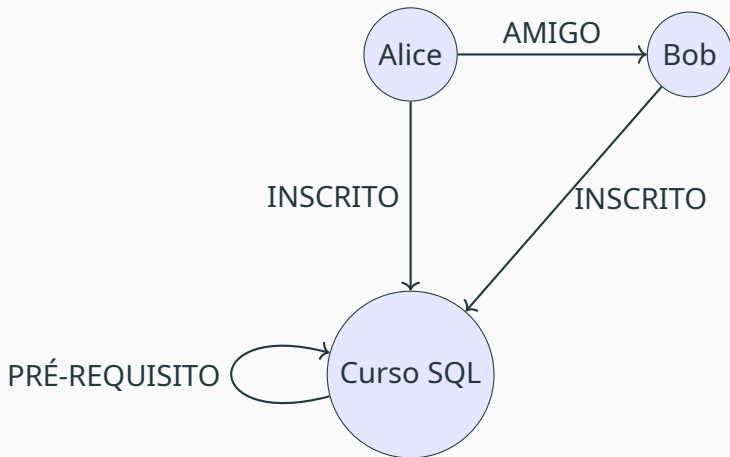
Armazena dados em famílias de colunas em vez de linhas.
Otimizado para escala horizontal massiva.

- **Analogia:** Um mapa 2D onde cada linha pode ter um conjunto diferente de colunas.
- **Pontos Fortes:** Elevada taxa de escrita, lida com petabytes de dados.
- **Uso:** Dados de séries temporais, logs históricos, análises em larga escala.
- **Exemplo:** **Apache Cassandra**, Google Bigtable.

Foca-se nas relações entre entidades. Os dados são representados como Nós e Arestas.

- **Analogia:** Uma rede social ou um mapa de rotas aéreas.
- **Pontos Fortes:** Consultas em relações complexas são muito mais rápidas que SQL Joins.
- **Uso:** Motores de recomendação, detecção de fraude, redes sociais.
- **Exemplo:** Neo4j, Amazon Neptune.

Visualizando uma BD de Grafos



Bases de Datos SQL Autónomas

O SQLite é uma biblioteca em linguagem C que implementa um motor de base de dados SQL pequeno, rápido, autónomo, de alta fiabilidade e completo.

- **Embarcado:** Não é um processo separado. Faz parte da aplicação.
- **Configuração Zero:** Sem necessidade de configuração ou administração.
- **Atómico:** Transações totalmente compatíveis com ACID, mesmo após falhas.
- **Multiplataforma:** O formato do ficheiro da base de dados é estável e portátil.

- **Arquitetura:**

- Usa **B-Trees** para armazenamento de dados em disco.
- Implementa **WAL (Write-Ahead Log)** para melhor concorrência.

- **Quando usar:**

- Armazenamento local para apps móveis/desktop.
- Dados intermédios durante análises.
- Websites de tráfego baixo a médio.

- **Quando evitar:**

- Ambientes com alta concorrência de escrita.
- Aplicações distribuídas em múltiplos servidores.

O H2 é uma base de dados relacional baseada em Java que pode ser usada em modo embarcado ou servidor.

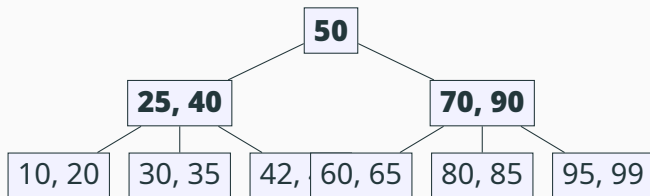
- **Performance:** Extremamente rápido porque pode correr inteiramente em RAM.
- **Padrão:** Suporta SQL standard e JDBC.
- **Compatibilidade:** Pode emular o comportamento de PostgreSQL, MariaDB ou Oracle.
- **Uso:** Testes unitários, prototipagem rápida e camadas de cache em aplicações Java.

SQL Avançado: Performance

Imagine uma tabela Alunos com 1.000.000 de registos.
Como é que o RDBMS encontra nome = 'Alice'?

- **Full Table Scan ($O(N)$):**
 - O motor lê todas as linhas, da primeira à última.
 - Se o registo estiver no fim, demora 1.000.000 de operações.
- **Index Scan ($O(\log N)$):**
 - Usando uma estrutura de dados ordenada (B-Tree).
 - Para 1.000.000 de registos, demora apenas ~ 20 operações.

Visualização de B-Tree



A procura começa na raiz e segue os ponteiros com base nos intervalos de chaves, reduzindo drasticamente o número de comparações.

```
-- Criar um índice padrão
CREATE INDEX idx_user_email ON users(email);

-- Índice composto (A ordem importa!)
CREATE INDEX idx_name_age ON users(last_name, first_name);

-- Índice único (Reforça lógica de negócio)
CREATE UNIQUE INDEX idx_student_id ON students(id_number);

-- Remover um índice
DROP INDEX idx_user_email;
```

Os índices não são “gratuitos”. Têm um custo:

- **Penalização de Escrita:** Cada vez que faz INSERT, UPDATE ou DELETE, o RDBMS também deve atualizar a estrutura do índice.
- **Espaço em Disco:** Os índices ocupam espaço significativo (por vezes tanto como os dados).
- **Manutenção:** Com o tempo, os índices podem tornar-se fragmentados e exigir reconstrução (REINDEX ou VACUUM).

Como saber se o seu índice está realmente a ser usado?

-- SQLite

EXPLAIN QUERY PLAN **SELECT** * **FROM** users **WHERE** email = 'a@b.com';

-- Output: SEARCH TABLE users USING INDEX idx_user_email (email=?)

-- PostgreSQL / MySQL

EXPLAIN ANALYZE **SELECT** * **FROM** users **WHERE** email = 'a@b.com';

- **Seq Scan:** Scan completo da tabela (Mau para tabelas grandes).
- **Index Scan:** Usando eficientemente o índice (Bom).

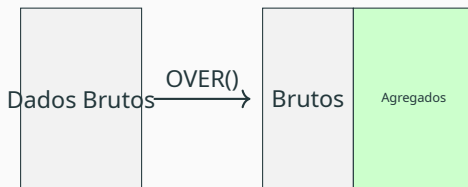
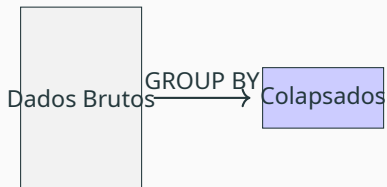
SQL Avançado: Analítica

Window Functions: Conceito

As Window Functions realizam um cálculo através de um conjunto de linhas da tabela que estão de alguma forma relacionadas com a linha atual.

- **Diferença Chave:** Ao contrário de GROUP BY, as linhas não são colapsadas. Cada linha preserva a sua identidade original.
- **Estrutura:** FUNCTION() OVER (PARTITION BY ... ORDER BY ...)
- **Funções:** SUM(), AVG(), RANK(), ROW_NUMBER(), LEAD(), LAG().

Visualizando Window Functions



Exemplos de Window Functions

```
-- Calcular um total acumulado
```

```
SELECT date, amount,  
        SUM(amount) OVER (ORDER BY date) as total_acumulado  
FROM sales;
```

```
-- Rankear itens dentro de uma categoria
```

```
SELECT name, category, price,  
        RANK() OVER (PARTITION BY category ORDER BY price DESC) as pos  
FROM products;
```

```
-- Lag/Lead (Aceder à linha anterior/seguinte)
```

```
SELECT date, amount,  
        LAG(amount) OVER (ORDER BY date) as valor_dia_anterior  
FROM sales;
```

Common Table Expressions (CTE)

Uma CTE fornece uma forma de definir um conjunto de resultados temporário que pode referenciar dentro de uma única consulta.

- **Sintaxe:** WITH nome_cte AS (SELECT ...)
SELECT ... FROM nome_cte;
- **Benefícios:**
 - **Legibilidade:** Divide consultas complexas em passos lógicos.
 - **Consultas Recursivas:** Permite consultar árvores ou dados hierárquicos.
 - **Modular:** Pode definir múltiplas CTEs numa única consulta.

Exemplo de CTE

```
WITH MediaDepto AS (  
    SELECT dept_id, AVG(salary) as sal_medio  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT e.name, e.salary, d.sal_medio  
FROM employees e  
JOIN MediaDepto d ON e.dept_id = d.dept_id  
WHERE e.salary > d.sal_medio;
```

- Aqui, calculamos primeiro as médias e depois usamos essa “tabela virtual” na consulta principal.

As CTEs recursivas são usadas para percorrer estruturas hierárquicas (como um organograma).

```
WITH RECURSIVE subordinados AS (  
  -- Inicial: Selecionar o CEO  
  SELECT id, name, manager_id FROM employees WHERE name = 'CEO'  
  UNION ALL  
  -- Recursivo: Juntar com subordinados  
  SELECT e.id, e.name, e.manager_id  
  FROM employees e  
  INNER JOIN subordinados s ON s.id = e.manager_id  
)  
SELECT * FROM subordinados;
```

Segurança de Bases de Dados

SQL Injection: O Ataque

A vulnerabilidade de base de dados mais comum. Ocorre quando o input do utilizador é concatenado numa string de consulta.

CÓDIGO VULNERÁVEL

```
username = input("Insira nome: ") # Input: ' OR '1'='1'
query = "SELECT * FROM users WHERE name = '" + username + "';"
# SQL resultante: SELECT * FROM users WHERE name = '' OR '1'='1';
```

- O atacante ignora a lógica e pode extrair, modificar ou apagar quaisquer dados.

Prepared Statements: O Escudo

Os Prepared Statements (ou Consultas Parametrizadas) garantem que o RDBMS trata o input do utilizador estritamente como dados, nunca como código.

```
# CÓDIGO SEGURO
username = input("Insira nome: ")
# 1. O template é enviado para a BD
# 2. Os dados são enviados separadamente como parâmetros
cursor.execute("SELECT * FROM users WHERE name = ?", (username,))
```

- **Fluxo de Trabalho:**

1. **Prepare:** A BD analisa, compila e otimiza o plano da consulta.
2. **Execute:** A BD vincula os valores ao plano e executa-o.

Além da segurança, os prepared statements oferecem benefícios de performance:

- **Protocolo Binário:** Os dados são enviados num formato binário nativo, reduzindo o overhead.
- **Reutilização de Plano:** O RDBMS não precisa de re-analisar a consulta de cada vez.
- **Segurança:** Trata automaticamente o escape de caracteres e tipos de dados.

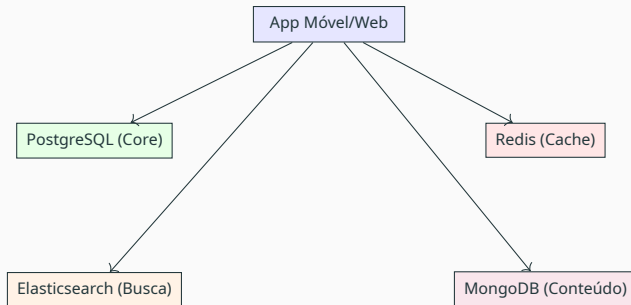
Arquiteturas Modernas

Persistência Poliglota

As aplicações modernas raramente usam apenas uma base de dados. Usam a melhor ferramenta para cada subproblema.

- **Encomendas (SQL):** Precisam de ACID para transações financeiras.
- **Catálogo de Produtos (NoSQL Documento):** Precisa de esquema flexível.
- **Procura de Produtos (Elasticsearch/Vector DB):** Precisa de procura de texto completo.
- **Sessão/Cache (Redis):** Precisa de latência inferior a milissegundos.
- **Ligações Sociais (Grafo):** Precisa de encontrar "Amigos de Amigos".

A Modern Data Stack



Resumo

- **Mudança de Paradigma:** SQL para estrutura e integridade; NoSQL para escala e flexibilidade.
- **Autónomas:** SQLite e H2 são ferramentas essenciais para muitos cenários.
- **Performance:** A indexação é o fator mais importante para a velocidade de leitura.
- **Analítica:** Window Functions e CTEs são ferramentas padrão para análise moderna.
- **Segurança:** Prepared statements são obrigatórios para qualquer aplicação.
- **Complexidade:** Os sistemas modernos são híbridos (Persistência Poliglota).